Linear Combinations

Introduction to Linear Combinations

Dear students, imagine you are designing an **AI-based color mixer app**. The app allows users to mix different base colors — red, green, and blue — to create any desired color.

Just like a DJ mixes tracks to get a new sound, or a digital artist blends brush strokes to paint something unique, your app works by mixing RGB values in different **proportions**.

These base colors — red (1,0,0), green (0,1,0), and blue (0,0,1) — are like **vectors**.

And when you mix them with different weights — say, 0.5 red, 0.2 green, and 0.3 blue — you get a new color.

This mixing process is what we call a **linear combination** in mathematics.

Definition: Linear Combination

If $\vec{w}$ is the vector in vector space V, then $\vec{w}$ is said to be a linear combination of the vectors $\vec{v}_1, \vec{v}_2, \dots\ \vec{v}_r$ in V if $\vec{w}$ can be expressed in the form

$$\vec{w} = k_1\vec{v}_1 + k_2\vec{v}_2 + \cdots + k_r\vec{v}_r$$

where $k_1, k_2, \dots, k_r$ are scalars. These scalars are called the coefficients of linear combinations.

Example: Let Red: $\vec{r} = (1,0,0)$, $Green$: $\vec{g} = (0,1,0)$, $Blue$: $\vec{b} = (0,0,1)$ in R^3. Then a color $\vec{c} = (0.3, 0.5, 0.2)$ can be written as linear combinations of $\vec{r}, \vec{g}, \vec{b}$

$$\vec{c} = (0.3, 0.5, 0.2)$$

$$\vec{c} = 0.3(1,0,0) + 0.5(0,1,0), 0.2(0,0,1)$$

$$\vec{c} = 0.3\vec{r} + 0.5\vec{g} + 0.2\vec{b}$$

Linear Combination (Geometrical view)

Let we have two vectors

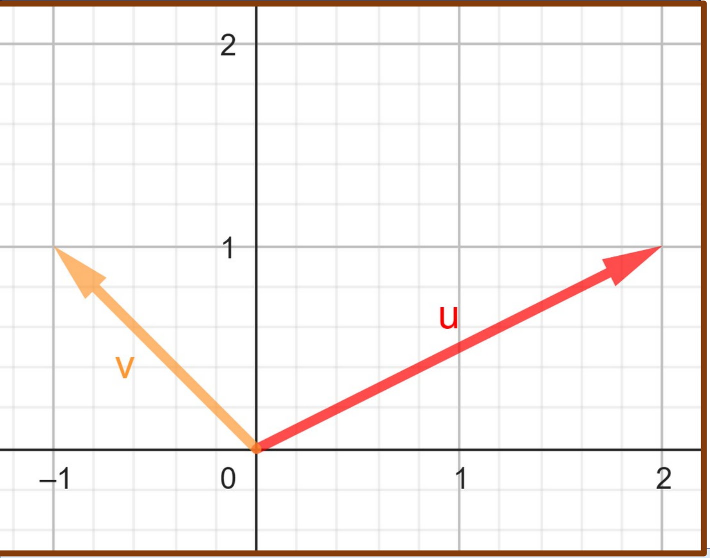
$\vec{u}$ and $\vec{v}$

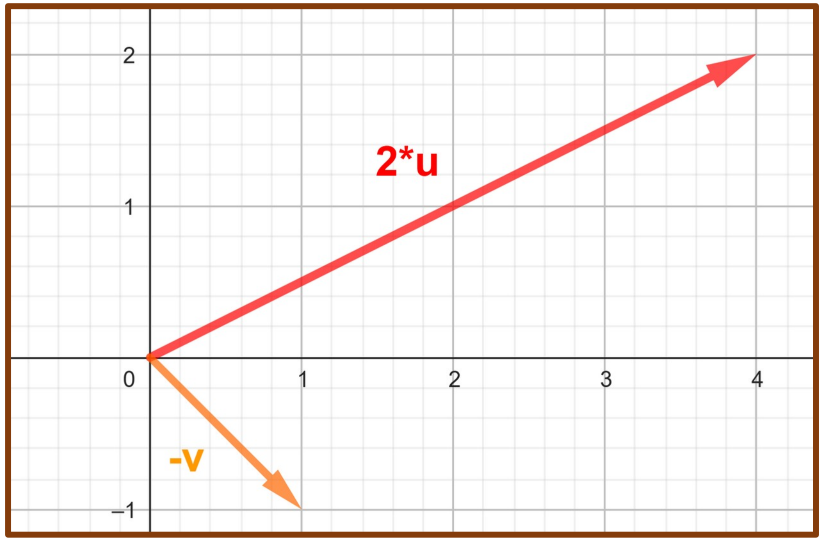
$$\vec{u} = \langle 2, 1 \rangle, \vec{v} = \langle -1, 1 \rangle$$

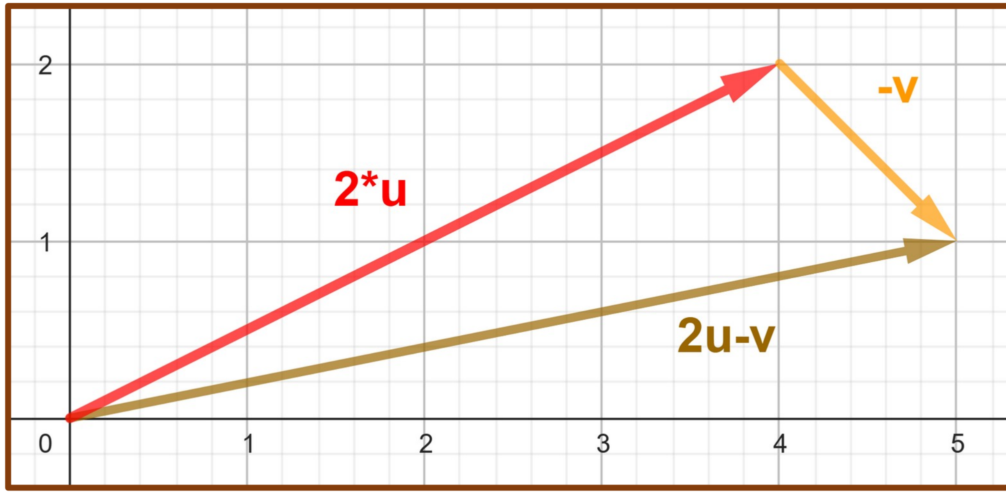
A Linear combination

$$2\vec{u} \ - \vec{v}$$

$$2\langle 2,1 \rangle - \langle -1,1 \rangle = \langle 5, 1 \rangle$$







Any time you're **scaling** two vectors and **adding** them like this, it's called a "linear combination" of those two vectors.

Example: Character Animation

Imagine you're developing a 3D game. You have two animation base poses (or movements) of a character: $\vec{u} = (1, 2, -1)$: Punch Forward and $\vec{v} = (6,4,2)$: Dodge Right

You want to generate a new pose: $\vec{w} = (9, 2,7)$: Punch while dodging

Can this new pose be created by **blending** the two base poses?

This is just like asking: can $\vec{w}$ be written as linear combination of $\vec{u}$ and $\vec{v}$.

In order to form linear combination of $\vec{u}$ and $\vec{v}$, there exist k_1 and k_2 such that

$$\vec{w} = k_1\vec{u} + \mathrm{k}_2\vec{v}$$

$$(9,2,7) = k_1(1, 2, -1) + k_2(6,4,2)$$

$$(9,2,7) = (k_1, 2k_1, -k_1) + (6k_2, 4k_2, 2k_2)$$

$$(9,2,7) = (k_1 + 6k_2, 2k_1 + 4k_2, -k_1 + 2k_2)$$

Solution:

Equating corresponding components gives:

$$k_1 + 6k_2 = 9, \qquad 2k_1 + 4k_2 = 2, \qquad -k_1 + 2k_2 = 7$$

Solving this system by using Gauss Elimination

$$\begin{bmatrix} 1 & 6 & : 9 \\ 2 & 4 & : 2 \\ -1 & 2 & : 7 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 6 & : & 9 \\ 0 & -8 & : & -16 \\ 0 & 8 & : & 16 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 6 & : 9 \\ 0 & 1 & : 2 \\ 0 & 1 & : 2 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 6 & : 9 \\ 0 & 1 & : 2 \\ 0 & 0 & : 0 \end{bmatrix}$$

$$\begin{cases} k_1 + 6k_2 = 9 & (1) \\ k_2 = 2 \\ 0 = 0 \end{cases}$$

Put $k_2 = 2$ in (1),
we get: $k_1 = -3$

So, $\vec{w} = -3\vec{u} + 2\vec{v}$
i.e.
$(9,2,7) = -3(1,2,-1) + 2(6,4,2)$

In game development terms:
The animator blends pose "Punch Forward" scaled by −3 (reverse?) and "Dodge Right" scaled by 2 to generate the “Punch While Dodging” animation.

Prove that $\vec{w}' = (4, -1, 8)$ is not linear combination of $\vec{u}$ and $\vec{v}$.

Equating corresponding components gives:

$$k_1 + 6k_2 = 4, \qquad 2k_1 + 4k_2 = -1, \qquad -k_1 + 2k_2 = 8$$

Solving this system by using Gauss Elimination

$$\begin{bmatrix} 1 & 6 & : & 4 \\ 2 & 4 & : & -1 \\ -1 & 2 & : & 8 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 6 & : & 4 \\ 0 & -8 & : & -9 \\ 0 & 8 & : & 12 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 6 & : & 4 \\ 0 & 1 & : & 9/8 \\ 0 & 8 & : & 12 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 6 & : & 9 \\ 0 & 1 & : & 2 \\ 0 & 0 & : & 3 \end{bmatrix}$$

This system has no solution.
So $\vec{w}'$
cannot be written as a linear combination of
$\vec{u}$ *and* $\vec{v}$

In game animation terms:
This pose cannot be generated using only the “Punch Forward” and “Dodge Right” animations — **you need a third motion** to achieve it!

Practice Problems

1. (Image Filter Weights)

An image filter uses three base filters represented by vectors:

Sharpen: $(2,-1,0)$, Blur: $(1,1,1)$, Edge Detect: $(0,-1,1)$

Can the combined filter $(3,-1,2)$ be written as a linear combination of these three?

2. In a neural network layer, the input vector is $\vec{x} = (2,3,1)$, and the weight vectors for 3 neurons are:

• $\overrightarrow{w_1} = (1,0,2)$, $\overrightarrow{w_2} = (0,1,-1)$, $\overrightarrow{w_3} = (2,2,2)$

The dot product output of each neuron is a linear combination.

Compute each output: $\boldsymbol{w_i} \cdot \vec{\boldsymbol{x}}$ and explain how these are linear combinations.

3. You have feature vectors $\overrightarrow{f_1} = (1,1,0)$, $\overrightarrow{f_2} = (0,1,1)$,

Can the new feature $\overrightarrow{f_3} = (2,3,2)$ be written as a linear combination of $\overrightarrow{f_1}$ and $\overrightarrow{f_2}$?

Home Work:

Exercise in Word File

14. Linear Combinations

THANKS